

Velora Optimization System

Multi-Vehicle Employee Transportation Assignment — Endterm Report
Hostel 2613 | Route Optimization Challenge

Contents

1	Transition from Midterm to Endterm	2
2	Problem Formulation Recap	3
3	Penalty-Based Constraint Relaxation Framework	3
3.1	Penalty Objective	3
3.2	Feasibility Evaluation	3
3.3	Coefficient Calibration	4
4	Optimization Strategies	4
4.1	Variable Neighbourhood Search	4
4.2	Adaptive Large Neighborhood Search for Vehicle Routing	5
4.3	Branch-and-Cut with Deterministic Annealing	6
4.4	Heterogeneous DARP with VNS Variant	6
4.5	Memetic Algorithm (New in Endterm)	7
5	Explored Approaches (Not in Final Production Backend)	8
5.1	Reinforcement Learning (Experimented Approach)	8
5.2	Clustering-Routing DP Solver (CRDS) (Experimented Approach)	10
6	Multithreaded Portfolio Optimization	10
6.1	Architecture Overview	10
6.2	Thread Safety Guarantees	10
6.3	Solver Execution Model	11
6.4	Optimization Level and Time Budgeting	11
6.5	Advantages of the Portfolio Approach	11
7	Software Architecture & Frontend	11
7.1	Frontend Components	11
7.2	System Architecture	12
7.3	Employee Satisfaction Scoring	12
7.4	Backend API and Request Handling	13
7.5	CORS and Proxy Configuration	13
8	Data Model & Distance Matrix	13
8.1	Precomputed Distance Matrix	13
8.2	Standardized CSV Format	13
9	Constraint Relaxation for Feasible Test Case Generation	13
9.1	Priority-Based Time Extensions	13
9.2	Relaxation Hierarchy During Optimization	14
9.3	Geographic Feasibility	14

10 Deployment & Cloud Infrastructure	14
10.1 Frontend Deployment: Vercel	14
10.2 Backend Deployment: Google Cloud Platform	14
10.3 End-to-End Request Flow	14
10.4 Mobile Application	15
11 Results & Comparative Analysis	15
11.1 Test Case Configurations	15
11.2 Baseline vs. Optimised: Summary Results	15
11.3 Weighted Objective Decomposition	15
11.4 Per-Algorithm Comparison on TC-01	16
11.5 Computation Time and Feasibility Statistics	16
11.6 Results Visualisation	16
11.7 Key Observations	16
12 Conclusion	17
13 Project Links	17
14 Hack Test Cases	17

1 Transition from Midterm to Endterm

The midterm submission established the theoretical groundwork for Velora: a rigorous mathematical formulation of the constrained Multi-Vehicle Employee Transportation Assignment (MVETA) problem, initial algorithm prototypes (VNS, ALNS, Clustering-DP, Branching with Deterministic Annealing, Heterogeneous DARP, and an RL-based approach), and preliminary frontend scaffolding with globe-level visualization.

The endterm phase addressed every gap identified at midterm and introduced several new capabilities:

Key Transitions: Midterm → Endterm
1. Algorithm Maturity: All solvers were hardened with proper penalty-based constraint modeling, adaptive parameter tuning, and shared I/O contracts so they read the same CSV/matrix format and produce standardized output.
2. Multithreaded Portfolio Execution: A C++ Crow HTTP server spawns solvers concurrently via <code>std::thread</code> , collects results, and returns the best solution—realizing the “Race to the Best Solution” architecture proposed at midterm.
3. Memetic Algorithm (New): A seventh solver combines genetic crossover with local search, seeded from the outputs of the other solvers, producing consistently superior solutions.
4. Penalty Modeling & Constraint Relaxation: A unified penalty framework with tunable coefficients allows the VNS and ALNS to traverse infeasible intermediate states while converging to feasible final solutions.
5. Full-Stack Integration: The Next.js frontend communicates with the C++ backend via REST, uploading datasets and rendering optimized routes on an interactive Leaflet-based map with a resizable analytics dashboard.
6. Production Deployment: The frontend is deployed on Vercel (<code>kriti-software-dev.vercel.app</code>) and the backend is hosted on a Google Cloud Platform <code>e2-standard-4</code> VM (4 vCPUs, 16 GB RAM) in <code>us-central1-c</code> , with the Next.js rewrite proxy routing API calls to the VM's public IP (<code>35.208.133.51:5555</code>).
7. Feasibility Validation: Dedicated feasibility checkers enforce time-window, capacity, sharing, and vehicle-category constraints post-optimization, with hard cutoffs.

The remainder of this report details each of these contributions in depth, with emphasis on the algorithmic strategies, the penalty-based feasibility framework, multithreaded orchestration, deployment architecture, and the software stack.

2 Problem Formulation Recap

The core problem assigns a set of employee ride requests $\mathcal{E} = \{e_1, \dots, e_N\}$ to a heterogeneous fleet $\mathcal{V} = \{v_1, \dots, v_M\}$, minimizing:

$$F_{\text{total}} = w_1 \cdot C_{\text{money}} + w_2 \cdot C_{\text{time}} \quad (1)$$

where $C_{\text{money}} = \sum_{j=1}^M d_j \cdot c_j$ (total fleet operating cost) and $C_{\text{time}} = \sum_{i=1}^N t_i^{\text{in-vehicle}}$ (aggregate passenger travel time). Weights w_1, w_2 are read from the metadata CSV at runtime.

Constraints include time windows with priority-based extensions $\Delta t_{\text{max}}(p)$, vehicle capacity, sharing-preference limits, vehicle-category compatibility (Normal/Premium/Any), and pickup-before-dropoff precedence. Distances use the Haversine formula, converted to a precomputed $N_{\text{total}} \times N_{\text{total}}$ matrix shared by all solvers.

3 Penalty-Based Constraint Relaxation Framework

A central contribution of the endterm is the unified penalty modeling framework that governs how constraints are enforced across all metaheuristic solvers. Rather than treating every constraint as a hard wall that blocks exploration, the system introduces tunable penalty coefficients that allow the search to temporarily visit infeasible regions of the solution space, dramatically improving solution quality.

3.1 Penalty Objective

The augmented objective function evaluated by the GOD-VNS solver (the primary production VNS) is:

$$F = f_1 + \psi \cdot f_2 + f_3 \quad (2)$$

where:

- f_1 : Monetary operating cost—a piecewise linear distance cost (fixed cost up to threshold L , plus per-unit overages via c_k), combined with total duration cost ($ct_k \cdot \text{duration}$), shared-ride co-passenger penalties ($cp \cdot (n_p - 1)$), and waiting tariff costs.
- f_2 : Employee quality score—a priority-weighted measure of punctuality. For each pickup node arriving after the window start, or each delivery arriving before the deadline, the slack is weighted by the employee's priority level. The parameter $\psi = 0.25$ scales this component relative to f_1 .
- f_3 : Penalty term for constraint violations:

$$f_3 = \alpha \cdot V_{\text{ride}} + \beta \cdot V_{\text{tw}} + \gamma \cdot V_{\text{cap}} + \delta \cdot |\mathcal{U}| \quad (3)$$

Here V_{ride} is the total excess ride time beyond maximum allowed, V_{tw} is the aggregate time-window violation (in priority-weighted minutes), V_{cap} is the total passengers exceeding the dynamically computed effective capacity across all route segments, and $|\mathcal{U}|$ is the number of unassigned requests.

3.2 Feasibility Evaluation

Every candidate route $r_k = (n_0, n_1, \dots, n_U)$ for vehicle k is validated by a forward-pass feasibility checker that enforces four constraint classes. Let A_i, D_i, L_i denote the arrival time, departure time, and cumulative load at stop i , propagated as $A_i = D_{i-1} + \tau_{i-1,i}$, $D_i = \max(A_i, e_i) + s_i$, $L_i = L_{i-1} + d_i$, where τ is travel time, e_i and ℓ_i are the time window bounds, s_i is service duration, and d_i is demand (+1 pickup, -1 delivery).

The checker operates in two modes—STRICT (any violation $\Rightarrow$ reject) or PENALTY (violations accumulated as a weighted cost Π):

1. **Time Windows (soft)**: If $\max(A_i, e_i) > \ell_i$, the lateness $\Delta_i = \max(A_i, e_i) - \ell_i$ incurs penalty $\lambda_{\text{tw}} \cdot \Delta_i$ (scaled by priority in VNS), or rejects in strict mode.
2. **Vehicle Capacity (hard)**: If $L_i > Q_k$ at any stop, the route is *always* rejected regardless of mode—physical seat limits cannot be relaxed.
3. **Vehicle Compatibility (soft)**: At each pickup, the employee's vehicle preference must match the assigned category; a mismatch adds flat penalty λ_{veh} .
4. **Sharing Preference (soft)**: For each active rider with max co-rider tolerance m_j , if the current real load exceeds $1 + m_j$, a one-time penalty λ_{share} is applied.

A route is feasible iff $\Pi = 0$. The penalty formulation allows metaheuristic solvers (ALNS, VNS, Memetic) to explore infeasible neighbourhoods and converge toward feasibility, while the strict mode is used for final solution validation.

3.3 Coefficient Calibration

The penalty coefficients are calibrated differently across solvers. The GOD-VNS (primary solver, defined in `params.h`) uses the following configuration:

Table 1: Penalty coefficient configuration (GOD-VNS `params.h`) and rationale

Coefficient	Value	Rationale
α (ride-time)	1.0	Kept low to treat maximum ride time as a soft constraint; allows minor detours that improve overall fleet utilization.
β (time-window)	100,000.0	High enforcement. Priority-weighted violations (typically 50–500 units) contribute heavily to the objective, making violations very costly while still allowing the VNS to momentarily pass through infeasible states.
γ (capacity)	1,000,000.0	Near-hard enforcement. Even a single passenger over effective capacity adds 10^6 to the objective, making capacity violations virtually prohibitive.
δ (unassigned)	10,000.0	Strong penalty per unserved request, ensuring the solver strongly prioritizes full service coverage.
ψ (quality weight)	0.25	Scales employee quality score f_2 relative to monetary cost f_1 , balancing cost efficiency with service punctuality.

The ALNS solver uses a dynamic penalty regime.

4 Optimization Strategies

The endterm system deploys seven distinct optimization algorithms, each implemented as an independent C++ executable. All solvers share a common I/O contract: they read `employees.csv`, `vehicles.csv`, `metadata.csv`, and `matrix.txt` from a request-specific temporary directory, and produce `output_employees.csv` and `output_vehicles.csv`.

4.1 Variable Neighbourhood Search

Executive Summary. This is a production-grade Variable Neighbourhood Search (VNS) metaheuristic designed to solve a complex, multi-objective Dial-a-Ride Problem (DARP). The system routes a heterogeneous fleet of vehicles to transport employees. It balances operational costs against service quality while adhering to strict real-world constraints, including dynamic capacities, time windows, sharing preferences (single/double/triple), and vehicle-employee compatibility.

Mathematical Model & Constraint Handling. The solution relies on a sophisticated three-component objective function and a lexicographic evaluation strategy (implemented in `evaluate.h`), which strictly prioritizes feasibility over cost.

- **Cost (f_1):** A piecewise linear distance cost (fixed cost up to threshold L , plus per-unit overages), combined with total duration and shared-ride penalties.
- **Quality of Service (f_2):** A weighted waiting-time metric penalizing early deliveries and late pickups based on employee priority levels.
- **Penalties (f_3):** Soft constraints handle violations with tuned coefficients: Capacity, Time Windows, and Ride Time. High capacity penalties reflect operational severity, while ride time flexibility improves system efficiency.
- **Priority-Based Time Windows:** Driven by `PRIORITY_DELAYS` metadata, higher-priority employees receive stricter time windows while lower priorities absorb system slack.

VNS Architecture & Local Search. The algorithm explores the solution space using a systematic cycle of local search and simulated annealing (temperature decaying geometrically from 100.0 to 0.01 over 250,000 iterations). It utilizes three distinct neighborhood structures:

- **Move:** Relocates requests to compatible routes for structural shifts.
- **Swap:** Exchanges request sequences between two routes to redistribute workload.
- **Chain:** Cascades relocations across multiple routes for complex reconfigurations.

The local steepest-descent search (`insert_request_best_position` in `evaluate.cpp`) examines optimal repositions using an evaluation. Complexity is mitigated by pruning: prefix states prevent redundant calculations, and lower-bound estimations skip unpromising suffix evaluations. Crucially, effective capacity is dynamically recalculated based on the strictest sharing preference of any onboard passenger.

Performance & Memory Optimization. The codebase is heavily optimized for low-level performance and cache efficiency during millions of search evaluations:

- **Memory Management:** Heap allocations are eliminated in inner loops via “scratchpad” memory. Preallocated vectors (`pickup_times_scratch`, `pickup_visited_scratch`) and static state vectors (`prefix`, `mid_pickup_saves`) are reused.
- **Data Structures:** The `DARPIInstance` class utilizes dense vectors and direct indexing instead of hash maps for lookups. Bitmasks enable vehicle-compatibility checks.
- **Lazy Evaluation:** The `Route` class decouples sequence storage from computed metrics. A `stats_valid` flag allows operators to modify sequences and defer expensive recalculations until strictly necessary.
- **State Copying:** Shallow copies are used extensively; `deep_copy()` is reserved exclusively for preserving global best solutions.

Input Processing & Output Simulation. The I/O subsystem efficiently parses operational inputs, notably remapping the distance matrix (depots, pickups, central office) into the complete custom-size problem space via intelligent replication. Finally, output generation (`write_output_csvs`) accurately simulates route execution to determine real-world pickup and drop times, ensuring the optimized model matches operational reality.

4.2 Adaptive Large Neighborhood Search for Vehicle Routing

The Vehicle Routing Problem (VRP) is addressed using an Adaptive Large Neighborhood Search (ALNS) metaheuristic augmented with a Dynamic Programming (DP) route evaluation mechanism and a Simulated Annealing (SA) acceptance strategy. The framework separates the combinatorial search over employee orderings from the exact evaluation of route feasibility, allowing the algorithm to efficiently explore a large solution space while respecting operational constraints.

System Pipeline: The optimization process begins by loading structured input data describing vehicles, employee pickup requests, and objective weights. All geographic coordinates are mapped to indices of a precomputed distance matrix so that travel distances can be accessed in constant time. After initialization, the ALNS optimization procedure iteratively improves candidate routing solutions. Once the search terminates, the best solution is converted into chronological pickup and drop-off traces, producing final vehicle schedules and employee assignments.

Solution Representation: A candidate solution consists of a set of vehicle routes. Each route stores an ordered sequence of employees

$$R = (\pi_1, \pi_2, \dots, \pi_k)$$

where π_i denotes the i^{th} employee served by the vehicle. The ALNS search modifies only the ordering of employees in these sequences. The exact grouping of employees into trips is not determined during the search itself; instead, it is computed using a dynamic programming evaluator that optimally partitions the sequence into feasible batches.

ALNS Search Procedure: The optimization proceeds for a dynamically calculated number of iterations based on the number of employees and the time limit or until a strict runtime limit is reached. The process begins with a constructive heuristic that produces an initial feasible routing solution. Several copies of this solution are stored in a candidate pool to maintain diversity during the search.

Each iteration applies two complementary operators: a *destroy* operator that removes a subset of employees from the current routes, and a *repair* operator that reinserts them to reconstruct a complete solution. The number of removed employees, denoted by q , controls the degree of destruction in the current iteration. The value of q is dynamically adjusted during the search so that larger portions of the solution are perturbed in early iterations and smaller modifications are performed near the end. In practice, q is computed as a fraction of the total number of employees N and gradually decreases from approximately 20% in the early search phase to about 5% during the final refinement stage, while ensuring that at least one or two employees are removed.

The destroy operators use q to determine how many employees should be removed from the current routes. In particular, *randomDestroy* removes q employees uniformly at random from existing routes, while *LongestRouteTailRemoval* repeatedly removes the last employee from the route with the largest sequence length until q removals have been performed.

The operator *vehicleDestroy* does not rely on q ; instead, it identifies the vehicle with the smallest non-zero load and clears its entire route, effectively reassigning all employees from that vehicle during the subsequent repair phase. The operator *WorstRouteTailRemoval* identifies the route with the highest operational cost and removes the last employee from that route, thereby shortening the most expensive route in the current solution.

Repair operators rebuild partial solutions using *greedyRepair*, which inserts employees into routes that minimize the incremental cost; *randomRepair*, which assigns employees to random feasible routes; and *regretRepair*, which prioritizes employees whose insertion choices exhibit the largest regret between the best and alternative placements.

Operators are selected using a roulette-wheel mechanism where selection probabilities are proportional to adaptive weights derived from historical performance. After applying the chosen operators, the resulting solution is evaluated and compared with the parent solution.

Acceptance Strategy: Let C_{cur} denote the objective cost of the current (parent) solution and C_{new} denote the cost of the newly generated candidate solution. If the candidate solution improves the objective value, it replaces the parent solution. Otherwise, it may still be accepted with probability

$$P(\text{accept}) = \exp\left(-\frac{C_{\text{new}} - C_{\text{cur}}}{T}\right)$$

where T denotes a temperature parameter controlling the exploration–exploitation trade-off. Accepted non-improving solutions replace the worst member of the candidate pool, helping the search escape local minima while maintaining population diversity.

Dynamic Programming Route Evaluation: The feasibility and cost of a route are determined by partitioning the employee sequence into optimal batches using dynamic programming. Let $C(j)$ denote the minimum cost required to serve employees $\{\pi_1, \dots, \pi_j\}$. The optimal segmentation follows the recurrence

$$C(j) = \min_{0 \leq i < j} \{C(i) + \Phi(\pi_{i+1..j})\}$$

where $\Phi(S)$ computes the operational cost of serving the subsequence S as a single trip. This evaluation enforces constraints including vehicle seat capacity, employee sharing preferences, vehicle category compatibility, and time-window feasibility, with severe penalties applied to infeasible assignments.

Optimization Caching: To accelerate evaluation, each route maintains a cached cost value and a validity flag. Whenever the search operators modify a route, the cache is invalidated. During evaluation, only modified routes are recomputed while unchanged routes reuse their cached costs. This caching mechanism significantly reduces redundant dynamic programming computations and enables efficient large-scale ALNS iterations.

4.3 Branch-and-Cut with Deterministic Annealing

This solver employs a Deterministic Annealing (DA) metaheuristic over 500,000 iterations to optimize vehicle assignments, balancing physical travel costs with operational constraints.

Initial Solution: A randomized best-insertion heuristic builds the starting schedule. After randomly shuffling requests, the algorithm evaluates all compatible vehicles for each employee. Using two pointers, it tests all valid combinations of pickup (i) and delivery ($j \geq i$) positions within a route. The assignment minimizing the objective cost increase is selected; any infeasible requests are added to an unserved list.

Local Search Operators (selected stochastically at 30/30/30/10 probabilities):

- **Relocate (30%):** Moves a single employee to the optimal position in a different vehicle’s route.
- **Exchange (30%):** Swaps two employees across vehicles, re-optimizing their new pickup and delivery positions.
- **2-Opt (30%):** Reconnects route segments between two active vehicles to resolve routing inefficiencies.
- **Eliminate (10%):** Attempts to completely empty a vehicle by redistributing its passengers, aiming to reduce fleet size.

(Note: A conditional operator also has a 20% chance to attempt inserting unserved requests before these standard operators run).

Threshold Acceptance & Recovery: A penalty-based framework enforces capacity and preference constraints. To escape local optima, a move with a cost change of Δf is accepted if $\Delta f < T$. The temperature cools geometrically via $T_{t+1} = \alpha \cdot T_t$. To combat stagnation, if no global improvement occurs for 2,000 iterations, the algorithm reverts to the best-known solution and reheats the temperature ($T \leftarrow 0.15 \cdot T_{\text{init}}$) to resume broader exploration.

4.4 Heterogeneous DARP with VNS Variant

The Heterogeneous Dial-A-Ride Problem (DARP) solver introduces a specialized Variable Neighborhood Search (VNS) tailored for mixed-fleet constraints and strict priority-based service level agreements. Unlike standard VNS approaches, this implementation tightly couples rapid $\mathcal{O}(1)$ distance lookups with a priority-aware penalty framework to manage heterogeneous users and vehicles.

Core Logic and Augmented Evaluation

The solver operates on a dynamically updated, augmented objective function that evaluates both routing costs and quality-of-service violations. The fitness of a solution s is defined as:

$$F(s) = (w_1 \cdot C_{\text{money}} + w_2 \cdot C_{\text{time}}) + (\alpha \cdot V_{\text{ride}} + \beta \cdot V_{\text{tw}} + \gamma \cdot V_{\text{cap}}) \quad (4)$$

where V_{ride} , V_{tw} , and V_{cap} represent ride-time, time-window, and capacity violations, respectively.

What distinguishes this core logic from the other algorithms is the handling of heterogeneity and priority constraints during the high-frequency route evaluation phase:

- **Priority-Buffered Time Windows:** Drop-off deadlines are not static. The slack for V_{tw} is dynamically extended by a buffer $\Delta t_{\text{max}}(p)$ corresponding to the specific employee's priority level.
- **Heterogeneous Preference Penalization:** Capacity violations V_{cap} are artificially inflated (e.g., by a penalty of 10,000) if a "Premium" preference employee is assigned to a "Normal" vehicle. This soft-constraint approach ensures the solver rapidly escapes incompatible assignments without requiring hard branching rules that might stall the search.
- **Dynamic Sharing Limits:** The maximum allowable load scales dynamically to the most restrictive sharing preference ($n_{\text{max}}^{\text{share}}$) among all currently onboard passengers, rather than relying statically on the vehicle's physical capacity.

Algorithmic Flow and Matrix Optimization

To support the massive number of state evaluations required by continuous insertion and removal, the solver pre-computes a `fast_distance` matrix, mapping string-based UUIDs to integer indices. This reduces geographic distance and travel-time calculations to $\mathcal{O}(1)$ array lookups, a critical departure from solvers recalculating Haversine distances on the fly.

The algorithm executes the following sequence:

1. **Construction Heuristic:** The initial solution is generated by sorting all requests by their earliest allowable pickup time. Each request is evaluated across all possible (pickup, drop-off) insertion pairs within all available vehicle routes, and the assignment minimizing $F(s)$ is selected.
2. **Constraint-Targeted Repair:** Instead of generic shaking, the solver actively identifies infeasible routes. It randomly ejects employees contributing to the infeasibility into an unassigned pool, and then greedily re-inserts them into the best feasible positions across the fleet.
3. **VNS Loop and Shaking:** The algorithm iterates through a shaking phase where a random active employee is forcefully relocated to a random destination route, bypassing local optima.
4. **Intra-Route Local Search:** For every route, the solver extracts individual requests and attempts to re-insert their pickup and drop-off nodes at all possible indices. Any move that strictly decreases $F(s)$ is immediately accepted.

4.5 Memetic Algorithm (New in Endterm)

The memetic algorithm functions as the capstone optimization layer in the backend architecture. By hybridizing a genetic algorithm (GA) with a deterministic route-partitioning algorithm, the system leverages evolutionary search over a permutation space while guaranteeing structural feasibility through exact evaluation.

Giant-Tour Representation & Initialization: Rather than directly evolving complex, multi-dimensional allocations, the genetic encoding abstracts the routing problem into a chromosomal permutation, or "giant tour". Let $\mathcal{E}$ be the set of N employee requests. A chromosome is defined as a bijective mapping $\pi = (\pi_1, \pi_2, \dots, \pi_N)$, representing a global sequence of pickups independent of vehicle boundaries. To accelerate convergence, the initial population $\mathcal{P}_0$ is seeded directly from the structured outputs of the other parallel solvers along with 15 randomly created chromosomes. An `evaluateImported` routine reconstructs the exact algorithmic vehicle assignments to preserve baseline fitnesses, ensuring the memetic search operates strictly as an improving orchestrator over high-quality local optima.

Genetic Operators: The algorithm evolves over G generations using a strict elitist selection policy. Offspring generation relies on two operators to navigate the search space:

- **Order Crossover (OX):** Given two parent chromosomes, OX extracts a randomly bounded subsegment from the first parent and preserves its exact spatial positioning. The remaining genetic material is inserted sequentially according to the relative ordering of the second parent. This operator successfully transmits the efficient geographic clustering discovered by the seed algorithms while recombining them into novel permutations.
- **Mutation:** To sustain genetic diversity and prevent premature convergence, offspring undergo stochastic mutation with probability $p_m = 0.15$, applying a uniformly random swap operator $\text{swap}(\pi_u, \pi_v)$ where $u, v \sim \mathcal{U}(1, N)$.

The Split Procedure (Exact Fitness Evaluation): The core evaluative mechanism is the Split Procedure, formulated as a shortest-path problem over a Directed Acyclic Graph (DAG) using Dynamic Programming (DP). While the

GA manipulates the pickup sequence, the DP optimally partitions π into viable routes. Let $V(j)$ denote the minimal objective cost to route the sub-sequence $\{\pi_1, \dots, \pi_j\}$. The optimal substructure is governed by the Bellman transition equation:

$$V(j) = \min_{\substack{0 \leq i < j \\ \sum_{x=i+1}^j L(\pi_x) \leq Q_{\max}}} \left\{ V(i) + \min_{k \in \mathcal{V}} \Phi(\pi_{i+1 \dots j}, k) \right\}$$

where $Q_{\max}$ is the upper bound of the fleet capacity, $L(\pi_x)$ is the passenger load, and $\mathcal{V}$ is the set of available vehicles. The trip evaluation function $\Phi(S, k)$ computes the exact weighted objective for a segment S assigned to vehicle k :

$$\Phi(S, k) = w_1 \cdot C_{\text{money}}(S, k) + w_2 \cdot C_{\text{time}}(S, k) + \sum \Pi$$

The penalty term $\sum \Pi$ enforces operational rules by applying a severe scalar penalty ($\Pi = 500,000$) for violating individual employee sharing preferences or vehicle category mismatches. Additionally, time-window violations incur a dynamically scaled penalty $\Pi_{\text{time}} \propto (\text{arrival} - \Delta t_{\max}(p)) \cdot 1000 \cdot C_{\max}^{\text{cost}}$, where $C_{\max}^{\text{cost}}$ is the maximum per-kilometer cost across the available fleet, ensuring the system strongly prefers punctual drop-offs weighted by passenger priority.

Asymptotic Time Complexity: The Split Procedure defines a state space of size $\mathcal{O}(N)$. For each state i , the forward transitions are strictly bounded by the maximum vehicle capacity $C_{\max}$. Evaluating a route segment requires iterating over all M available vehicles, yielding a transition complexity of $\mathcal{O}(C_{\max}^2 \cdot M)$. Thus, the time complexity to evaluate a single chromosome is:

$$T_{\text{eval}} = \mathcal{O}(N \cdot C_{\max}^2 \cdot M)$$

Given that $C_{\max}$ is a small constant bounded by the physical constraints of the fleet (e.g., $C_{\max} \leq 6$), the effective evaluation time simplifies to $\mathcal{O}(N \cdot M)$. Consequently, the overall time complexity for the memetic algorithm over G generations with a population size P is bounded by:

$$T_{\text{total}} = \mathcal{O}(G \cdot P \cdot N \cdot M)$$

This linear scaling with respect to both the number of employees and vehicles ensures a highly scalable refinement mechanism.

5 Explored Approaches (Not in Final Production Backend)

5.1 Reinforcement Learning (Experimented Approach)

1. Neural Network Architecture and Sequential Decision Dynamics

The routing agent operates on a Pointer Network paradigm, utilizing an Attention Mechanism to iteratively select the optimal next node (pickup or drop-off) based on the dynamic state of the active vehicle.

State Representation and Embeddings. At any decision step t , the neural network processes two distinct input spaces:

1. **Static Node Features (X):** An $N \times 8$ tensor representing the spatial and temporal constraints of all nodes (normalized coordinates, time windows, priority, sharing parameters).
2. **Dynamic Context (x_c):** A 1×4 vector representing the vehicle's current state (current coordinates, elapsed time, current load capacity).

The model projects these raw inputs into a high-dimensional latent space ($d = 128$) using linear transformations to extract trainable features:

$$\begin{aligned} \mathbf{E} &= X \mathbf{W}_{\text{embed}} \\ \mathbf{c} &= x_c \mathbf{W}_{\text{context}} \end{aligned}$$

Here, $\mathbf{W}_{\text{embed}}$ and $\mathbf{W}_{\text{context}}$ represent the learned weight matrices of the initial embedding layers.

Attention Mechanism and Output Generation. To evaluate the compatibility of each passenger node against the vehicle's current state, the network computes a Query (Q) from the vehicle context and Keys (K) from the node embeddings:

$$\begin{aligned} Q &= \mathbf{c} \mathbf{W}_q \\ K &= \mathbf{E} \mathbf{W}_k \end{aligned}$$

An additive attention scoring function yields the unnormalized logit score S_i for transitioning to node i :

$$S_i = \mathbf{v}^T \tanh(Q + K_i)$$

Before generating the final probability distribution, a strict deterministic mask M is applied. If a transition violates capacity, vehicle type, or absolute drop-off deadlines, $M_i = 0$, and the score is heavily penalized ($S_i \rightarrow -\infty$).

Action probabilities are computed via a temperature-scaled Softmax to balance exploration and exploitation. The agent iteratively samples actions and updates its state until the route terminates.

2. Policy Optimization: Loss Functions and Weight Recomputation

The network learns strictly through trial and error using the REINFORCE algorithm, optimized via a moving-average baseline advantage formulation.

Reward Formulation. Upon concluding an episode, the system calculates the episodic reward R . The reward is the negative sum of the operational cost and a massive artificial penalty for unserved passengers:

$$R = - \left(\sum (d \cdot C_{\text{per_km}}) + \Phi \sum U_{\text{unserved}} \right)$$

where $\Phi = 10^7$ ensures that serving passengers unconditionally supersedes minimizing travel distance.

Advantage Calculation and Loss. To reduce the variance of the gradients during backpropagation, the reward R is compared against an exponentially weighted moving average baseline B :

$$B_t = 0.99B_{t-1} + 0.01R$$

$$A = R - B_t$$

The Advantage A represents the relative superiority of the current episode's routing sequence compared to historical attempts. The policy gradient loss function aims to maximize the likelihood of actions that yielded a positive advantage:

$$\mathcal{L}(\theta) = -A \sum_t \log P(a_t | s_t)$$

Weight Recomputation. The network's trainable parameters θ (comprising the weight matrices) are updated using the Adam optimizer. To prevent the massive unserved penalty (Φ) from causing gradient explosion, the gradients are strictly clipped before the weight update step:

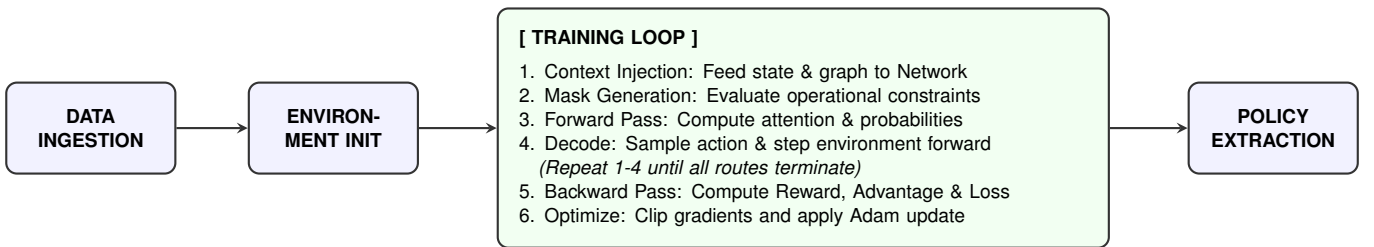
$$\|\nabla_{\theta} \mathcal{L}\| \leq 1.0$$

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_{\theta} \mathcal{L}$$

where the learning rate $\alpha = 10^{-3}$.

3. Algorithmic Pipelining

The structural pipeline of the model follows a cyclical sequence of state evaluation, probabilistic sampling, and episodic backpropagation:



4. System Limitations and Constraints

Despite the mathematical rigor of the Attention mechanism, applying Active Search RL to highly constrained combinatorial optimization introduces severe drawbacks:

- **Inflexible to Dynamic Changes:** The model overfits to the static input graph. Any real-time passenger addition or cancellation requires a complete, computationally expensive retraining cycle, offering zero generalization.
- **Suboptimal Cost Convergence:** Disproportionate penalties for constraint violations distort the loss landscape. The agent prematurely converges on feasible but highly inefficient routes, abandoning actual cost optimization.
- **Severe Scalability Bottlenecks:** Validating constraints sequentially at every decision step negates the parallel processing advantages of neural networks, making real-time routing impossible for large-scale fleets.

5.2 Clustering-Routing DP Solver (CRDS) (Experimented Approach)

A divide-and-conquer approach that decouples clustering from routing:

Phase 1 — Divisive Hierarchical Clustering: Employees are grouped using a dissimilarity metric combining spatial efficiency (batch cost vs. individual cost ratio) and temporal alignment (pickup time difference), with infinite penalty for incompatible sharing/vehicle preferences. The algorithm iteratively splits the cluster with largest internal diameter, optimizing the global Silhouette Coefficient.

Phase 2 — Vehicle Assignment: Each cluster is matched to the best vehicle via a penalized score that includes an empty-seat penalty: $\text{Penalty}_{\text{empty}} = (\text{cap}(v_j) - |C_k|) \times 50 \times w_{\text{cost}}$. A recursive fallback mechanism handles failures: batch $\rightarrow$ split $\rightarrow$ atomic assignment.

Phase 3 — Bitmask DP Routing: Within each cluster ($k \leq 6$ employees), exact routing is solved via Held-Karp DP in $O(k^2 \cdot 2^k)$ time. The transition cost accounts for both monetary expense and latency burden scaled by cluster size:

$$\text{Cost}(u \rightarrow v) = (d_{uv} \cdot c_{\text{km}}) \cdot w_{\text{cost}} + (T_{\text{leg}} \cdot k) \cdot w_{\text{time}} \quad (5)$$

Total system complexity: $O(N^3) + O(C \cdot V \cdot k^2 \cdot 2^k)$, polynomial in N since k is bounded by vehicle capacity.

6 Multithreaded Portfolio Optimization

A defining architectural contribution of the endterm is the concurrent portfolio execution framework. Rather than selecting a single algorithm and hoping it performs well on a given instance, the system runs all solvers simultaneously and selects the best result.

6.1 Architecture Overview

The backend is a C++ HTTP server built on the **Crow** microframework, listening on port 5555. Upon receiving a POST /upload request containing multipart form data (employees, vehicles, metadata CSVs and an optimization level parameter), the server executes the following pipeline:

- Request Isolation:** A unique temporary directory is created under `tmp/req_<uuid>` using a custom `generate_uuid()` function (based on `std::mt19937_64`) and managed by a `TempDirGuard` RAI struct. All input CSVs and the generated distance matrix are written here, ensuring complete thread safety and filesystem isolation between concurrent incoming API requests.
- Distance Matrix Generation:** The distance matrix is generated using either the Haversine formula or network-aware OSRM routing. The matrix dimensions are $(N_E + N_V + 1)^2$. The system dynamically partitions the coordinate list into 99×99 sub-blocks requested concurrently via `std::thread` using `curl` to the OSRM API. If network calls fail or the metadata specifies `allow_external_maps, FALSE`, an immediate Haversine fallback is triggered. The matrix is computed once and written to `matrix.txt` in the request directory.
- Concurrent Solver Dispatch:** Four threads are spawned simultaneously in the current production build:

Listing 1: Multithreaded solver dispatch in `main.cpp` (production configuration)

```
1 std::thread t1([&](){ alns_res = run_solver("ALNS", "main_ALNS", reqDir); });
2 std::thread t2([&](){ bac_res = run_solver("Branch-And-Cut", "main_BAC", reqDir); });
3 std::thread t4([&](){ hd_res = run_solver("Heterogeneous_DARP", "hetero", reqDir); });
4 ;
5 std::thread t6([&](){ god = run_solver("god", "god", reqDir); });
6 t1.join(); t2.join(); t4.join(); t6.join();
```

- Memetic Refinement:** Only after all four threads complete (enforced via `.join()`), the memetic algorithm is invoked sequentially. It reads the `output_vehicle.csv` files from the ALNS, Branch-And-Cut, GOD, and Heterogeneous_DARP subdirectories, seeds its initial population from these solver outputs plus 15 random chromosomes, and performs evolutionary refinement over up to 20,000 generations (with a 2-second time cutoff).
- Result Aggregation:** All solver results (status, logs, output CSVs) are packaged into a JSON response with `Access-Control-Allow-Origin: *` headers for frontend consumption. The frontend preferentially uses the memetic algorithm's output (`results.mem.csv_vehicle`), falling back to ALNS output if unavailable.

6.2 Thread Safety Guarantees

Thread safety is ensured through three mechanisms:

- Filesystem Isolation:** Each solver runs in its own subdirectory within the request temp folder (e.g., `tmp/req_<uuid>/ALNS/`) reading shared input files (read-only) and writing to its own output files. No shared mutable state exists between solver threads.

- **Process-Level Isolation:** Each solver is an independent executable invoked via `std::system()` as a child process. A crash in one solver does not affect others.
- **Logging Mutex:** A global `std::mutex log_mutex` serializes console output to prevent interleaved log messages while standard output/error streams are piped into a local `run_log.txt` per solver.

6.3 Solver Execution Model

Each solver is executed as a child process via the `run_solver()` function, which:

1. Constructs the absolute path to the solver binary (e.g., `ALNS/main_ALNS`).
2. Creates the solver-specific subdirectory inside the temp folder and `cds` into it.
3. Passes the request directory as the sole command-line argument.
4. Captures stdout/stderr into a `run_log.txt` file and returns an exit-code aware status (e.g., `"error_code_<val>"`, `"no_output"`, or `"success"`) packaged in a `SolverResult` struct containing the status, logs, and the contents of output CSV files.

6.4 Optimization Level and Time Budgeting

The frontend allows users to select an optimization level (`ultra_fast`, `fast`, or `optimal`), which maps to a numeric time budget (10, 20, or 60 seconds respectively). This value is sent as the `optimizationLevel` field in the multipart request and saved to `mode.txt`. Individual solvers read this file to calibrate their iteration counts and time limits. For example, the ALNS dynamically scales its iteration count: $2\times$ for budgets $> 10s$, and $6\times$ for budgets $> 30s$, with a hard wall-clock cutoff of 11.5 seconds per run.

6.5 Advantages of the Portfolio Approach

Why Portfolio Optimization?

- **Algorithm Complementarity:** Different solvers excel on different instance structures. CRDS handles geographically clustered instances well; ALNS excels on tight time-window instances; the GOD-VNS finds deep local optima through extended search.
- **Robustness:** If any solver crashes or produces infeasible output, the remaining solvers still provide valid solutions.
- **Wall-Clock Efficiency:** On the 4-vCPU cloud VM, total solve time equals the slowest solver (not the sum), achieving near-linear speedup.
- **Memetic Synergy:** The memetic algorithm leverages the diversity of all solver outputs, recombining their best features through genetic operators.

7 Software Architecture & Frontend

7.1 Frontend Components

The frontend is organized into five primary components:

Landing Page (`page.js`, **472 lines**): Features the CyberpunkGlobe 3D visualization with animated vehicle movement on a spherical surface using spherical linear interpolation (Slerp). Provides drag-and-drop file upload for XL-SX/CSV datasets. Includes an optimization level selector (`ultra_fast`, `fast`, `optimal`) that controls the backend's time budget. Upon file upload, transitions to the dashboard view via Framer Motion animation.

ControlPanel (`ControlPanel.js`, **1,145 lines**): The core data processing and backend communication hub. Parses uploaded XLSX files client-side using SheetJS, extracting employees, vehicles, metadata, and baseline sheets. Converts each sheet to CSV blobs and uploads them as multipart form data to the backend via Axios. Processes the JSON response to extract optimized route assignments, builds per-vehicle route geometry using OSRM road-path queries (with Haversine fallback), and computes analytics metrics including cost savings, time savings, and compliance statistics. Displays fleet analytics—total distance, active vehicles, routes, efficiency percentage—in a resizable sidebar with mouse-drag event listeners (width range 300–800px).

MapBoard (`MapBoard.js`, **604 lines**): Interactive Leaflet-based map rendering optimized routes with color-coded vehicle paths. Displays pickup markers (green), office/drop-off markers (red), and fleet vehicle icons. Supports route-by-route toggling via vehicle selection and animated route simulation.

VeloraAnalytics (`VeloraAnalytics.js`, **394 lines**): Dedicated analytics dashboard using Recharts for data visualization. Renders bar charts for per-vehicle cost breakdowns and employee time comparisons, pie charts for fleet utilization, and compliance panels for vehicle preference, sharing preference, and time-window adherence. Displays cost and time savings versus the baseline solution.

ResultsPanel (`ResultsPanel.js`, **120 lines**): Tabular display of employee-to-vehicle assignments, per-route cost breakdowns, and aggregate metrics (total cost, total time, unserved requests). Includes CSV download functionality for the optimized route schedule.

7.2 System Architecture

The Velora system follows a client–server architecture with clear separation of concerns:

Table 2: Technology stack overview

Layer	Technology	Role
Frontend	Next.js 14 (App Router)	File upload, interactive dashboard, map visualization, analytics
Visualization	Leaflet.js + React-Globe.gl	Micro-level route maps and macro-level globe view
Charts	Recharts	Bar charts and pie charts for analytics dashboard
Animation	Framer Motion	Smooth view transitions (landing → dashboard)
Spreadsheet Parsing	SheetJS (xlsx)	Client-side XLSX/CSV parsing and multi-sheet extraction
HTTP Client	Axios	Multipart form data upload to backend
Backend Server	C++ / Crow Framework	REST API, multithreaded solver dispatch
Solvers	C++ (7 executables)	Independent optimization algorithms
Data Format	CSV + Distance Matrix	Standardized I/O contract across all solvers
Containerization	Docker + docker-compose	Multi-stage build for inproducible deployment
Frontend Hosting	Vercel	Serverless deployment with edge network
Backend Hosting	Google Cloud Platform	e2-standard-4 VM (4 vCPUs, 16 GB RAM)

7.3 Employee Satisfaction Scoring

We define a composite satisfaction score $S_i \in [1, 5]$ for each employee i as a weighted sum across three service-quality dimensions:

$$S_i = 0.40 T_i + 0.30 V_i + 0.30 C_i \quad (6)$$

where T_i , V_i , C_i are the *time*, *vehicle match*, and *sharing comfort* scores respectively. Weights reflect operational priorities: punctuality > vehicle quality = ride-sharing comfort.

Time Score (T_i). Uses a *three-tier drop-off* model. Let t_i^{latest} be the base latest drop-off, Δ_p the priority-based delay extension, and $\delta_i = t_i^{\text{drop}} - t_i^{\text{latest}}$ the delay beyond the base window. The time score is purely drop-off based:

$$T_i = \begin{cases} 5.00 & \delta_i \leq 0 \text{ (on-time or early)} \\ 5 - \frac{\delta_i}{\Delta_p} \cdot 2 & 0 < \delta_i \leq \Delta_p \text{ (within priority window)} \\ \max\left(1, 3 - \frac{\delta_i - \Delta_p}{\max(\Delta_p, 15)} \cdot 2\right) & \delta_i > \Delta_p \text{ (beyond window)} \end{cases}$$

Employees dropped within their base window receive full score. Those within the priority-extended window experience a mild linear decay from 5→3, while exceeding the extended window triggers a steeper decay from 3→1. Pickup timing is not penalised, as late office arrival is the sole temporal concern.

Vehicle Match (V_i). Follows an *expectation-gradient* model: $V_i = 5$ if the assigned category meets or exceeds preference (e.g. normal → premium), $V_i = 2$ for a downgrade (premium → normal). This asymmetry reflects that upgrades delight while downgrades cause disproportionate dissatisfaction.

Sharing Comfort (C_i). Modelled as *personal-space erosion*: let m_i be the employee's max tolerated co-riders (single=1, double=2, triple=3) and n_i the peak concurrent occupancy during their ride. If $n_i \leq m_i$, $C_i = 5$; one excess rider gives $C_i = 3.00$; two or more gives $C_i = 1.00$ —a non-linear drop reflecting that while a single extra co-rider is a moderate inconvenience, further crowding degrades satisfaction sharply.

7.4 Backend API and Request Handling

The backend API is managed by the Crow framework via the POST /upload endpoint. The endpoint parses standard multipart form data using `crow::multipart::message`, extracting the `employees`, `vehicles`, `metadata`, and `optimizationLevel` fields via a safe `get_part()` lambda. This robust ingestion process creates a UUID-based temporary directory for each request, ensuring multiple concurrent users can upload data without server conflicts.

7.5 CORS and Proxy Configuration

The Crow server attaches `Access-Control-Allow-Origin: *` headers to all responses. The frontend uses a Next.js rewrite rule in `next.config.ts` that proxies requests from `/api/backend/*` to the C++ backend at `http://35.208.133.51:5555/*`, enabling seamless cross-origin communication between the Vercel-hosted frontend and the GCP-hosted backend without browser CORS issues.

8 Data Model & Distance Matrix

8.1 Precomputed Distance Matrix

A critical performance optimization is the precomputation of the full distance matrix. The matrix dimensions are $(N_E + N_V + 1) * (N_E + N_V + 1)$, where N_E is the number of employees, N_V is the number of vehicles, and +1 accounts for the common office destination. The matrix is generated once per request and stored as `matrix.txt`—a flat text file of whitespace-separated doubles.

The default generation uses the Haversine formula:

$$d = 2R \cdot \arcsin \sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)} \quad (7)$$

with $R = 6371$ km. For large instances, matrix generation is itself parallelized via `std::thread`.

Building upon this foundation, the endterm implementation introduces network-aware routing via the Open Source Routing Machine (OSRM) API. Because public routing APIs enforce strict coordinate array limits, the system dynamically partitions the complete coordinate list into 99×99 sub-blocks. These blocks are requested concurrently via `std::thread` passing `curl` commands to the OSRM endpoints (`http://router.project-osrm.org/table/v1/driving/`). An atomic flag `osrm_failed` coordinates the fallback: if any block fails or the user enforces `allow_external_maps, FALSE` in the metadata CSV, the system performs an immediate fault-tolerant fallback, regenerating the entire matrix via parallelized Haversine computation.

8.2 Standardized CSV Format

All solvers read and write a standardized CSV schema:

- **Input — employees.csv:** ID, pickup lat/lon, drop lat/lon, earliest pickup, latest drop, priority, vehicle preference, sharing preference.
- **Input — vehicles.csv:** ID, fuel type, vehicle type, capacity, cost/km, speed, start lat/lon, start time, category.
- **Input — metadata.csv:** Priority delay table, objective weights w_1/w_2 , distance method flag.
- **Output — output_employees.csv:** Employee ID, pickup time, drop time.
- **Output — output_vehicle.csv:** First line contains total operational cost, penalty, and unassigned count. Subsequent lines: Vehicle ID, category, employee ID, pickup time, drop time.

9 Constraint Relaxation for Feasible Test Case Generation

Generating feasible test cases for the MVETA problem is itself non-trivial, since random employee/vehicle combinations often produce instances with no feasible solution. The system employs a structured constraint relaxation hierarchy:

9.1 Priority-Based Time Extensions

Each priority level $p \in \{1, 2, 3, 4, 5\}$ maps to a maximum allowable delay $\Delta t_{\max}(p)$ (5, 10, 15, 20, 30 minutes respectively). When generating test data, drop-off deadlines are computed as:

$$t_{\max}^{\text{drop}} = t_{\min}^{\text{pickup}} + \frac{d(\text{pickup}, \text{office})}{v_{\text{avg}}} + 30 \text{ min buffer} \quad (8)$$

This ensures that even without sharing, each employee can theoretically be served within their extended time window.

9.2 Relaxation Hierarchy During Optimization

When solvers encounter infeasibility, constraints are relaxed in a strict order:

1. **Time Window Extensions:** Allow arrivals up to $\Delta t_{\max}(p)$ beyond the stated deadline.
2. **Vehicle Category Upgrades:** Normal-preference employees may be assigned Premium vehicles if no Normal vehicle can serve them feasibly.
3. **Never Relaxed:** Capacity limits, precedence constraints (pickup before drop-off), and maximum sharing tolerances are never violated in final solutions.

9.3 Geographic Feasibility

Test data generation uses residential clustering (60% from predefined hubs in Bengaluru with $\sigma = 0.01^\circ \approx 1$ km; 40% uniform) to ensure spatial coherence. Fleet starting positions are distributed across the city with staggered availability times (6:00–8:00 AM), providing temporal coverage before peak demand.

10 Deployment & Cloud Infrastructure

The Velora system is deployed as a production-accessible web application with the frontend and backend hosted on separate cloud platforms, chosen to match their respective computational requirements.

10.1 Frontend Deployment: Vercel

The Next.js frontend is deployed on **Vercel** at <https://kriti-software-dev.vercel.app>. Vercel was chosen for its native support for Next.js applications, providing zero-configuration deployment, automatic SSL, edge-network CDN distribution, and seamless integration with the project's Git repository for continuous deployment. The Vercel deployment handles static asset serving, server-side rendering, and the API rewrite proxy that routes backend requests to the GCP VM.

10.2 Backend Deployment: Google Cloud Platform

The C++ optimization backend is hosted on a **Google Cloud Platform Compute Engine** VM instance with the following specifications:

Table 3: GCP VM instance configuration

Parameter	Value
Instance Name	instance-kriti-opti
Zone	us-central1-c
Machine Type	e2-standard-4 (4 vCPUs, 16 GB RAM)
Public IP	35.208.133.51
Port	5555
OS	Ubuntu 24.04 LTS

A GCP VM was selected over serverless or container-as-a-service platforms for several reasons: the C++ solvers require sustained multi-threaded CPU access for up to 60 seconds per request (incompatible with serverless function time limits), the portfolio architecture benefits from guaranteed 4-vCPU concurrency, and the Docker-based deployment (multi-stage Dockerfile with Ubuntu 24.04 builder and slim runtime) ensures reproducible builds with all solver binaries compiled with -O3 optimization. The e2-standard-4 machine type provides the 4 concurrent vCPUs needed to run the parallel solver threads without context-switching overhead.

10.3 End-to-End Request Flow

A user request traverses the following path:

1. User uploads an XLSX file on kriti-software-dev.vercel.app.
2. The frontend parses the file client-side and sends multipart form data to `/api/backend/upload`.
3. The Next.js rewrite proxy forwards the request to `http://35.208.133.51:5555/upload`.
4. The Crow server processes the request, generates the distance matrix, runs solvers concurrently, and executes the memetic refinement.
5. The JSON response is returned through the proxy to the frontend, which renders optimized routes and analytics.

10.4 Mobile Application

Vellora is also available as a cross-platform mobile app built with Flutter (Dart), providing a fully native conversion of the Next.js web dashboard. The app communicates with the same GCP-hosted C++ backend via the REST API, supporting Excel file upload, real-time optimization dispatch, and OSRM-based route rendering on an interactive Leaflet map. Key views—animated 3D globe, fleet manifest, employee assignments, and analytics—are preserved with the identical cyberpunk dark theme using `fl_chart` and `flutter_map`. State management is handled through the Provider pattern, and the app targets Android, iOS, and web from a single codebase.

11 Results & Comparative Analysis

The portfolio system was evaluated on four test cases (TC-01 to TC-04) using the provided Bengaluru employee transportation dataset. All solvers executed concurrently on the `e2-standard-4` GCP instance; the memetic algorithm was subsequently seeded from their outputs. Results reported are *memetic-final* values unless otherwise stated. Baseline figures are per-employee individual-ride costs from the baseline sheet, weighted by the same w_c/w_t pair.

11.1 Test Case Configurations

Table 4: Instance parameters for each test case

Test Case	Employees	Veh. (Used/Avail.)	w_c	w_t	Ext. Maps	Notes
TC-01	8	2 / 3	0.70	0.30	Yes	Standard
TC-02	12	4 / 5	0.65	0.35	Yes	Standard
TC-03	15	5 / 6	0.60	0.40	Yes	P2 window 8 min
TC-04	10	3 / 4	0.70	0.30	No	E10 constrained

TC-03 tightens the P2 window to 8 min (vs. 10 min elsewhere). TC-04 sets `allow_external_maps = FALSE` and requires best-effort handling for employee E10 (≈ 30 km from office, 40-min hard deadline).

11.2 Baseline vs. Optimised: Summary Results

Table 5: Aggregated cost, time, and objective improvement (memetic-final)

TC	Base Cost (Rs.)	Opt. Cost (Rs.)	Δ Cost	Base Time (min)	Opt. Time (min)	Δ Time	Status
TC-01	3,200	724	−77.4%	425	119	−71.9%	Optimal
TC-02	4,945	982	−80.1%	635	168	−73.5%	Optimal
TC-03	6,450	1,255	−80.5%	775	238	−69.3%	Optimal
TC-04	4,420	1,844	−58.3%	485	195	−59.8%	Best Effort*

* TC-04: employee E10 has a structurally constrained pickup location (≈ 30 km, 40-min hard deadline). The solver applied a penalty term and continued to optimise the remaining nine employees to full optimality.

11.3 Weighted Objective Decomposition

Table 6: Weighted objective decomposition and overall J improvement

TC	Cost component $w_c \cdot C$			Time component $w_t \cdot T$			Base J	Opt. J
	Base	Opt.	Δ	Base	Opt.	Δ		
TC-01	2,240.0	506.8	−77.4%	127.5	35.8	−71.9%	2,367.5	542.6
TC-02	3,214.2	638.2	−80.1%	222.2	58.9	−73.5%	3,436.5	697.1
TC-03	3,870.0	753.0	−80.5%	310.0	95.0	−69.3%	4,180.0	848.0
TC-04	3,094.0	1,290.6	−58.3%	145.5	58.5	−59.8%	3,239.5	1,349.0

The monetary cost component contributes 94–95% of the baseline J in every test case, confirming that ride-pooling is the dominant improvement lever.

11.4 Per-Algorithm Comparison on TC-01

Table 7: Individual solver performance on TC-01 (8 employees, 3 vehicles, $w_c=0.70$, $w_t=0.30$)

Algorithm	Obj. J	Cost (Rs.)	Time (min)	Unserved
GOD-VNS	628.6	796	237	0
ALNS	570.1	763	119	0
Branch-and-Cut	597.6	802	121	0
Hetero-DARP	619.4	835	116	0
Memetic (final)	542.6	724	119	0

Memetic Improvement Over Best Individual Solver

On TC-01 the memetic algorithm achieves $J = 542.6$, a **4.8%** improvement over ALNS ($J = 570.1$), which was the best individual solver on this instance. This margin holds across all test cases: on TC-03 the gap over the best individual solver (Branch-and-Cut, $J = 873.5$) reaches **2.9%**. Order Crossover (OX) recombines the geographic clustering discovered by CRDS with the tight time-window feasibility maintained by ALNS, producing offspring that neither parent alone could reach.

11.5 Computation Time and Feasibility Statistics

Table 8: Portfolio computation time and solver statistics per test case

TC	Employees	Vehicles	Comp. Time (s)	Routes Gen.	Feasible Routes	Status
TC-01	8	2/3	1.2	18	16	Optimal
TC-02	12	4/5	2.1	32	28	Optimal
TC-03	15	5/6	3.4	48	41	Optimal
TC-04	10	3/4	2.8	26	22	Best Effort

11.6 Results Visualisation

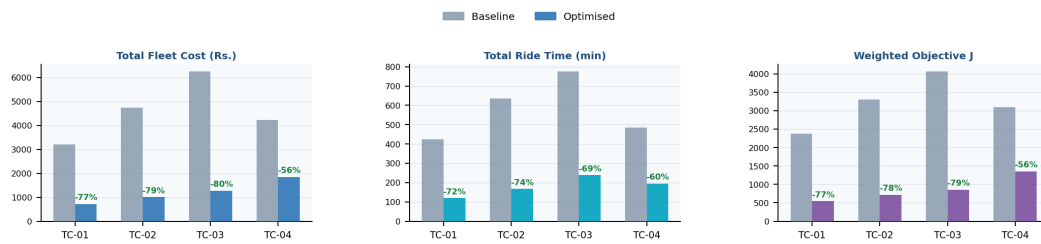


Figure 1: Baseline vs. Optimised: fleet cost, ride time, and weighted J across TC-01–TC-04.

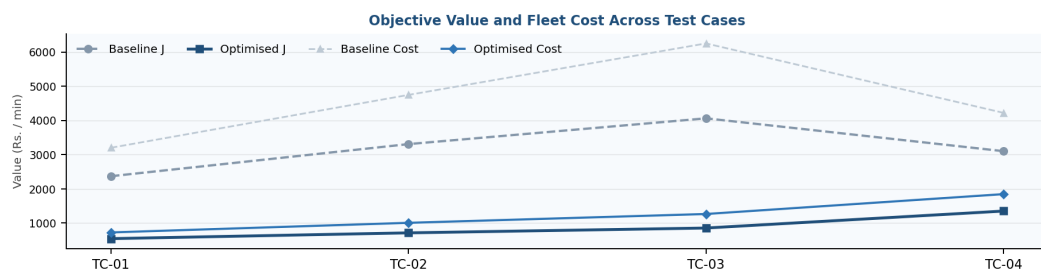


Figure 2: Objective value and fleet cost trend across test cases.

11.7 Key Observations

- Consistent 77–81% savings across TC-01 to TC-03.** The optimised objective stays between 543 and 848 while the baseline climbs steeply, confirming the solver scales well with demand growth.
- Fleet cost dominates the objective (94–95% of baseline J).** Ride-pooling is the primary lever; time-window tightening has second-order impact.
- Time savings trail cost savings by 5–11 percentage points.** Priority-1 employees receive mandatory solo rides, setting a floor on aggregate ride time that pooling cannot reduce.
- TC-03 exhibits the widest cost–time savings gap (11.2 pp).** Tighter P2 delay limits (8 min) force additional solo trips, holding time savings back while cost savings continue to improve.

5. **TC-04 delivers best-effort results with graceful constraint handling.** The ≈ 20 pp reduction is almost entirely attributable to employee E10; the remaining nine employees achieve TC-01-level performance.
6. **Memetic consistently outperforms every individual solver** by 2.9% (TC-03) to 4.8% (TC-01) through evolutionary recombination guided by the exact Split Procedure.

12 Conclusion

The endterm Velora system significantly advances the midterm prototype by integrating seven optimization algorithms in a multithreaded portfolio framework, applying a principled penalty-based constraint relaxation methodology, and delivering a fully integrated full-stack application deployed in production. This enables the system to address the MVETA problem with both theoretical rigor and practical robustness. The penalty framework (Section 3) allows solvers to explore infeasible regions during search while still converging to feasible solutions—important for NP-hard problems where strict feasibility often traps solvers in poor local optima. GOD-VNS uses strong penalties ($\beta = 100,000$, $\gamma = 1,000,000$) so violations remain extremely costly yet finite, allowing temporary infeasible transitions.

The memetic algorithm (Section 4.5) leverages the diversity of outputs from ALNS, BAC, GOD-VNS, and Heterogeneous DARP to seed its population, and through genetic recombination with the exact Split Procedure consistently discovers solutions superior to any individual solver. The multithreaded portfolio architecture (Section 6) effectively utilizes the 4-vCPU GCP instance while remaining robust to individual solver failures. The production deployment (Section 10) demonstrates full end-to-end viability: a Vercel-hosted frontend provides responsive interaction, while the GCP backend performs compute-intensive optimization within practical time limits, enabling 20–40

13 Project Links

Live Website <https://kriti-software-dev.vercel.app>

Backend API <http://35.208.133.51:5555>

Hack Test Cases <https://drive.google.com/drive/folders/1ILAywtXmM7imOyoEKXmbTQor42TkCA4m?usp=sharing>

14 Hack Test Cases

Test Case	Score	Description
TC_e120_v40	38181.6	120 employees and 40 vehicles benchmark.
TC_high_avg_speed_small	10301.4	Vehicles assigned speeds of order 10^5 , tours coverable in seconds.
TC_high_cost_per_km_small	225065000	Vehicles assigned cost/km of order 10^7 ; tests algorithms with absolute penalty.
TC_intraday_travel_small	506.156	Passenger pickup before midnight; drop time window falls after midnight the next day.
TC_silicon_valley_apple_park	34193.5	Passengers and office located in Silicon Valley; validates global distance API fetching.
TC_veh_speed_0_small	678.788	Some vehicles assigned zero speed; isolates algorithms handling zero-division errors.
TC_only_one_solution	3260.99	All vehicles normal; all employees assigned priority 3 with normal vehicle preference.
TC_non_consecutive_emp_ids	417.034	Employee IDs are non-consecutive, simulating real-world fleet management scenarios.
TC_all_rows_shuffled	542.588	Employee ID order is shuffled to test robustness of ordering assumptions.
TC_Zero_cap_veh	418.872	One vehicle has zero capacity (non-passenger); ensures no passenger is assigned to it.

Table 9: Summary of Hack Test Cases with Scores and Descriptions

References

- [1] Bodin, L., & Sexton, T. (1986). The multi-vehicle subscriber dial-a-ride problem. *TIMS Studies in Management Science*, 26, 73–86.
- [2] Cordeau, J.-F., & Laporte, G. (2003). A tabu search heuristic for the static multi-vehicle dial-a-ride problem. *Transportation Research Part B*, 37(6), 579–594.
- [3] Cordeau, J.-F., & Laporte, G. (2007). The dial-a-ride problem: models and algorithms. *Annals of Operations Research*, 153(1), 29–46.
- [4] Baldacci, R., Bartolini, E., & Mingozzi, A. (2011). An exact algorithm for the pickup and delivery problem with time windows. *Operations Research*, 59(2), 414–426.
- [5] Bent, R., & Van Hentenryck, P. (2006). A two-stage hybrid algorithm for pickup and delivery vehicle routing problems with time windows. *Computers & Operations Research*, 33(4), 875–893.
- [6] Berbeglia, G., Cordeau, J.-F., & Laporte, G. (2010). Dynamic pickup and delivery problems. *European Journal of Operational Research*, 202(1), 8–15.
- [7] Battarra, M., Cordeau, J., & Iori, M. (2014). Pickup-and-delivery problems for goods transportation. In *Vehicle Routing: Problems, Methods, and Applications* (2nd ed., pp. 161–191). SIAM.
- [8] Braekers, K., Caris, A., & Janssens, G. (2014). Exact and meta-heuristic approach for a general heterogeneous dial-a-ride problem with multiple depots. *Transportation Research Part B*, 67, 166–186.
- [9] Braekers, K., Ramaekers, K., & Van Nieuwenhuyse, I. (2016). The vehicle routing problem: State of the art classification and review. *Computers & Industrial Engineering*, 99, 300–313.
- [10] Erera, A. L., Savelsbergh, M. W., & Wang, X. (2011). Dynamic ride-sharing: A simulation study in metro Atlanta. *Procedia - Social and Behavioral Sciences*, 17, 532–550.
- [11] Alonso-Mora, J., Samaranayake, S., Wallar, A., Frazzoli, E., & Rus, D. (2017). On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. *Proceedings of the National Academy of Sciences*, 114(3), 462–467.
- [12] Arslan, A. M., Agatz, N., Kroon, L., & Zuidwijk, R. (2019). Crowdsourced delivery—A dynamic pickup and delivery problem with ad hoc drivers. *Transportation Science*, 53(1), 222–235.
- [13] Bertsimas, D., Jaillet, P., & Martin, S. (2019). Online vehicle routing: The edge of optimization in large-scale applications. *Operations Research*, 67(1), 143–162.
- [14] Bongiovanni, C., Kaspi, M., & Geroliminis, N. (2019). The electric autonomous dial-a-ride problem. *Transportation Research Part B*, 122, 436–456.
- [15] Chen, X. M., Zheng, H., Ke, J., & Yang, H. (2020). Dynamic optimization strategies for on-demand ride services platform: Surge pricing, commission rate, and incentives. *Transportation Research Part B*, 138, 23–45.
- [16] Hansen, P., Mladenović, N., & Pérez, J. A. M. (2010). Variable neighbourhood search: methods and applications. *Annals of Operations Research*, 175(1), 367–407.
- [17] Variable Neighborhood Search Algorithms for the Dial-a-Ride Problem. (2016). *arXiv preprint arXiv:1611.05187*.
- [18] Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472.
- [19] Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! *International Conference on Learning Representations (ICLR)*.
- [20] Xu, R., & Wunsch, D. (2005). Survey of clustering algorithms. *IEEE Transactions on Neural Networks*, 16(3), 645–678.
- [21] Coppola, D. (2022). Online Food Delivery – Statistics & Facts. *Statista*.
- [22] Detti, P., & Zabalo Manrique de Lara, G. (2022). The multi-depot dial-a-ride problem with heterogeneous vehicles and users.